コンピュータシステム2

第3回 プロセス管理とスケジューリング

神戸大学 DX・情報統括本部 情報基盤センター
大学院 システム情報学研究科
工学部 情報知能工学科
鳩野逸生・大野麻子

本日の講義内容

1.プロセスとは

2.マルチプログラミング

3.プロセスのメモリにおけるレイアウト

4.プロセスの操作・状態遷移

5.CPUスケジューリング（ FCFS, SJF, SRTF, RR ）

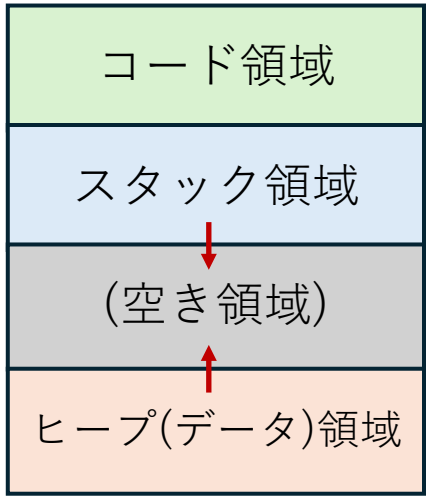
6.プロセスとスレッド

（参考）C言語のソースプログラムとコンパイル結果

プロセス (Process) の定義

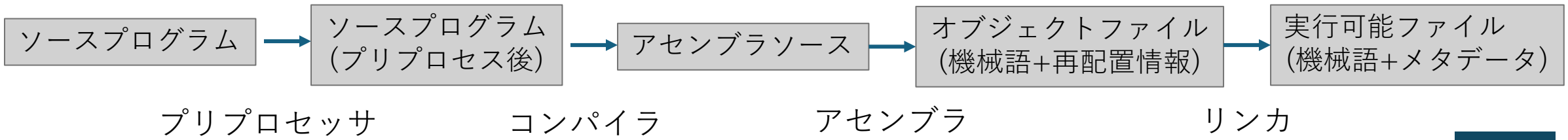
- “動作中のプログラム”であり，メモリ上の構造を持つ
- task（タスク）とも呼ばれる
- CPU スケジューリングの対象となる基本単位
- OSが資源（リソース）を割り当てる論理的な対象

・・・他，多くの定義が与えられている



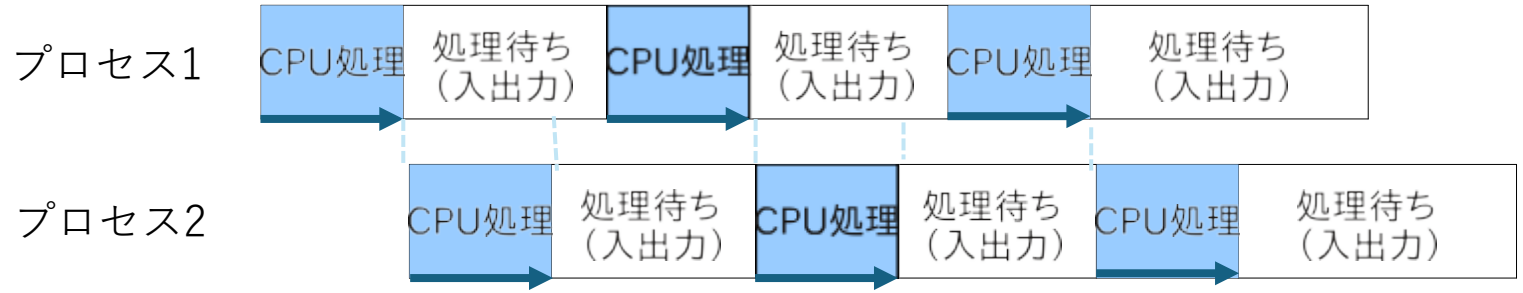
プロセスとプログラム

- この場合のプログラムとは？
  - C / C++ / Java 等のソースファイル?
  - コンパイルされたオブジェクトファイル?
  - 実行のためメモリ中にロードされた一連の機械語
- メモリ上にどのようにロードされるか
  - コード領域，ヒープ (データ) 領域，スタック領域
- C のソースプログラム / アセンブラソースプログラムと各領域？
  - プログラムの動作理解のためにも必要



マルチプログラミング

- 現代の計算機 (PC 含む) では**本質的に並列に**処理が行われている (必要がある)
- 通常の計算機では CPU 利用率は低い
  - CPU の処理時間に比べてI/O 待ち時間が長い
- CPU の利用効率を上げるために複数の**プロセスを並行して動作**させることを**マルチプログラミング**とよぶ
- 場合や目的に適したCPU 処理の割り当てが必要
  - CPUスケジューラが複数のプロセス間でCPUを切り替え，有効利用 (**並行処理＝並列処理**)



プロセス生成の概要:プロセスはどのように作られるのか？

- OSがプログラムをメモリにロードし，
  実行単位（プロセス）を作成
- 新しいプロセスは既存のプロセス（親）によって生成される
- Unix系では，fork() システムコールで新しいプロセスを生成
- 親プロセスは複数の子プロセスを
  持つことができる
- システム全体ではプロセスツリー
  （階層構造）を形成する

```
pid_t pid = fork();
if (pid == 0) {
    // 子プロセスの処理
} else {
    // 親プロセスの処理
}
```

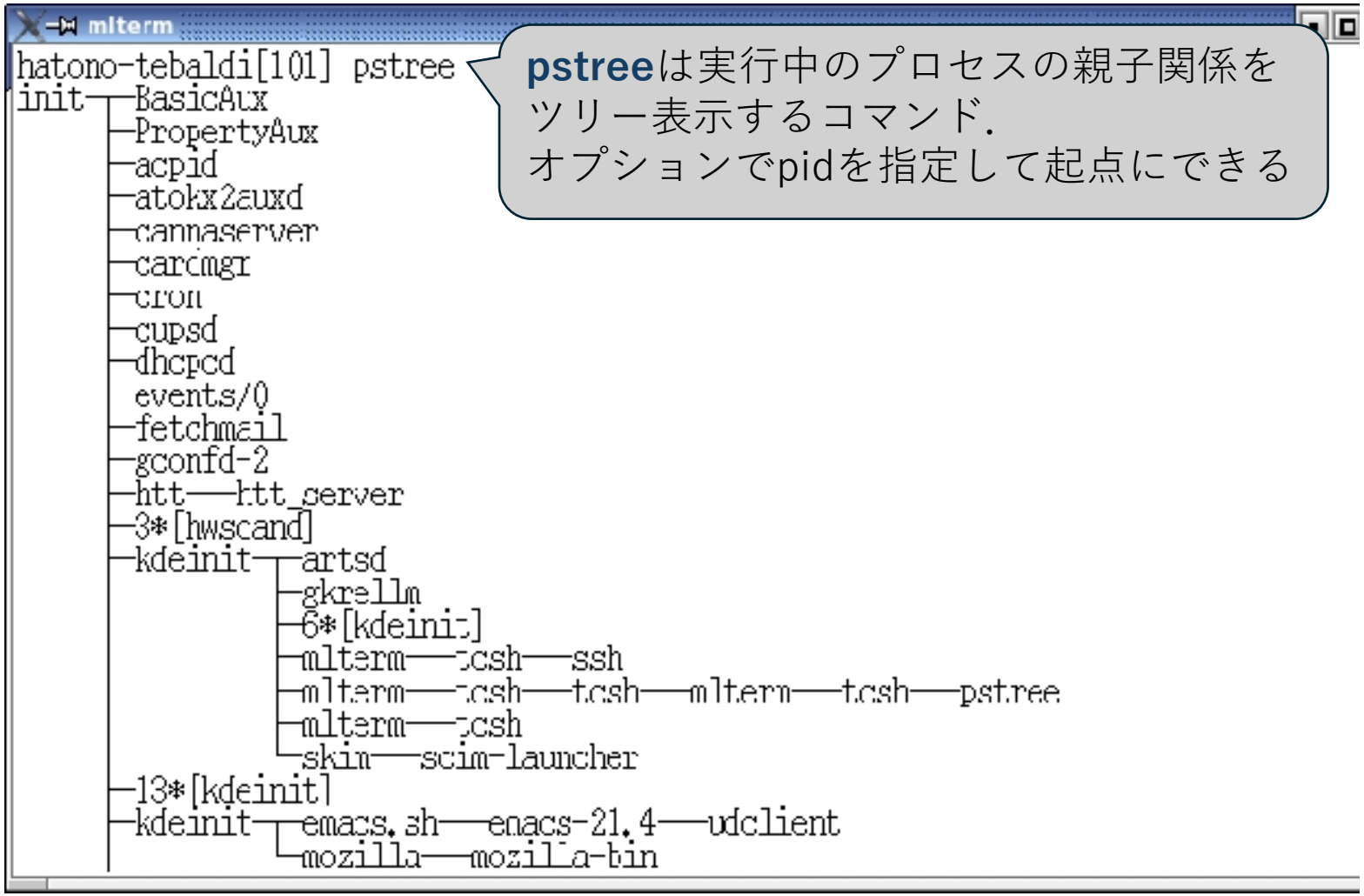
生成時の処理の流れ

- fork() 実行の内部動作
  - OSが新しいPCB（Process Control Block）領域を確保
  - 親プロセスのコード・データ・スタック領域をコピー
  - 子プロセスに新しいPID（Process ID）を割り当て
  - 親・子ともに同じ命令位置から再開（戻り値で区別）

項目	親プロセス	子プロセス
PID	既存	新規に割り当て
コード領域	共有	共有
データ領域	コピー	コピー
スタック	コピー	独立

プロセスの階層

- 親 (parent)プロセスと子(child)プロセスは1対多のツリー構造

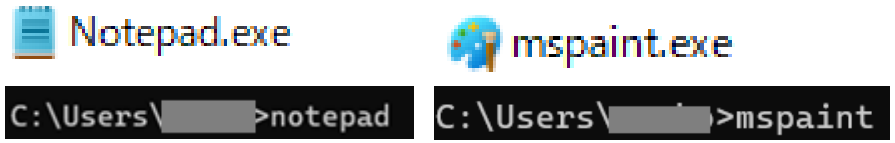


pstreeは実行中のプロセスの親子関係をツリー表示するコマンド.
オプションでpidを指定して起点にできる

wslで試せます

プロセスの起動・終了

- ユーザによる起動
  - GUI のメニュー・アイコンから起動
  - Shell / Command prompt からコマンド入力



- ユーザによる終了
  - 終了メニュー選択
  - ウインドウ操作
- OS による起動
  - システム起動時に実行
  - イベント発生時に起動

プロセスの中断と再開

- プロセスの中断 (suspend)
  - システム障害の発生，デバッグ時，システム高負荷時など
  - 自ら (実行中状態の場合)またはシステム，他のプロセスにより行われる
  - 資源の解放 (主記憶は直ちに)

- プロセスの再開 (resume)
  - 中断した時点の命令からの再開
  - システムあるいは他のプロセスにより行われる

- プロセスの消滅 (terminate)
  - 実行終了 → 利用していた資源を解放
  - 実行中 → 使用している資源の情報を各種キューやテーブルから削除
  - 対応するPCBの領域をシステムに返却

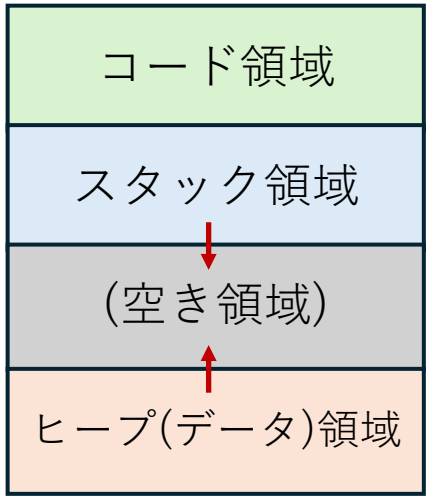
プロセスを確認するコマンド

- MacOSX / Linux (ターミナル)
  - ps, w
  - top
  - vmmap
- Windows (コマンドプロンプト)
  - tasklist (Ctrl-Shift-Escでタスクマネージャを起動でもよい)
- コマンドを実行してみよ
  - 意味は, 次のようにして調べることができる
    - man コマンド名 (MacOSX / Linux)
    - help コマンド名 (Windows)

Windowsの場合はWindow+R → cmd や wsl（上記全てではない）で試せます

プロセスのメモリレイアウト

- プログラムとしてメモリ上にロードされる時のレイアウト:



コード領域には命令

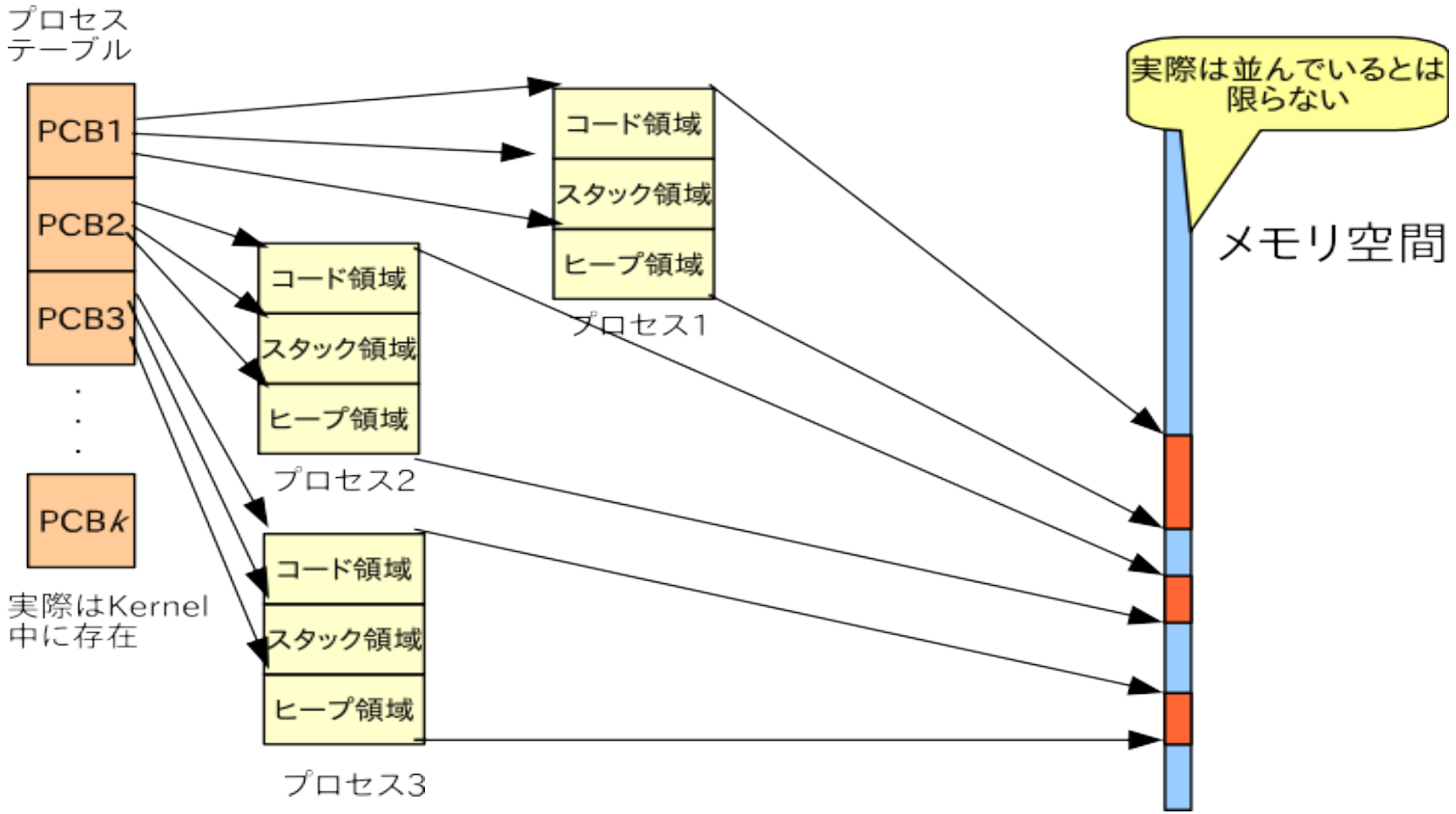
スタック領域には一時的な変数

ヒープ領域にはデータ

  - プロセスコントロールブロック (PCB) で管理される

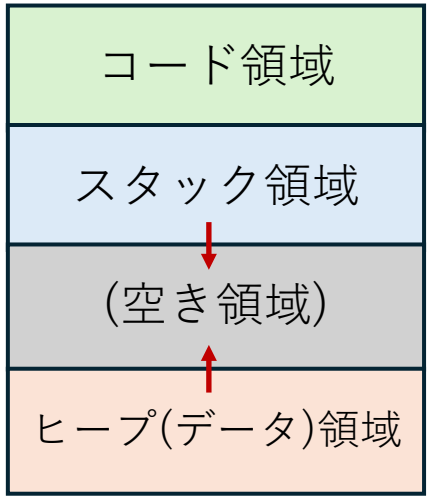
プロセス（メモリ上の配置）

- Process TableのEntryであるPCBによってプロセスを管理



プロセスの操作

- プロセスの生成
  1. PCB用のメモリ領域を確保
  2. プロセスをロードする領域を確保
     - コード領域，スタック領域，ヒープ領域，
  3. あるプロセス (親) が新しいプロセス (子) を生成
     - 代表的なのがUnixのforkシステムコール
       親子プロセス間でコード領域を共有，データ領域とスタック領域をコピー
     - スレッドの実現にも使用
       親子プロセス間でコード領域とデータ領域を共有，スタック領域をコピー



プロセス制御ブロック

- **プロセス制御ブロック（Process Control Block: PCB）**
  - システム内部でプロセスを表すデータ構造．Process table entry とも呼ぶ
- PCBの項目:
  - 次のPCBへのポインタ
  - プロセス識別子(PID)
  - プロセスの状態
  - プロセスの優先度
  - 親プロセス
  - プログラムのコード領域へのポインタ
  - プログラムのデータ領域へのポインタ
  - プログラムのスタック領域へのポインタ
  - **プログラムカウンタの退避領域**
  - **レジスタの退避領域**
  - メモリ管理情報
  - 入出力情報
  - ファイル管理情報

プロセスの切り替え処理

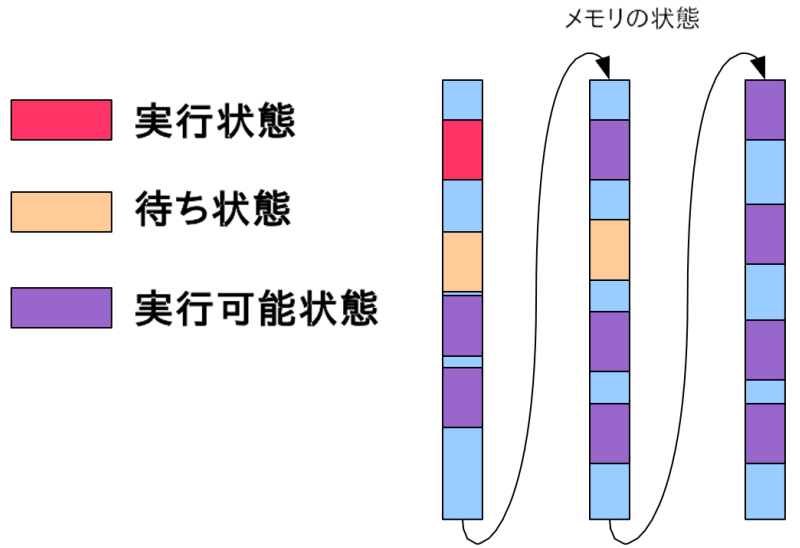
- **コンテキスト・スイッチ**（文脈切り替え）
  - CPUが実行するプロセスを切り替える一連の処理

    文脈 → プロセスの実行状態の記録
  - プログラムカウンタ，レジスタ等の再開に必要な情報を
    メモリ内のPCB領域に転送
    - プログラム状態語 (PSW: Program Status Word)
  - CPUスケジューラを呼び出し次に実行するプロセスを決定し，
    PSWをCPUに転送

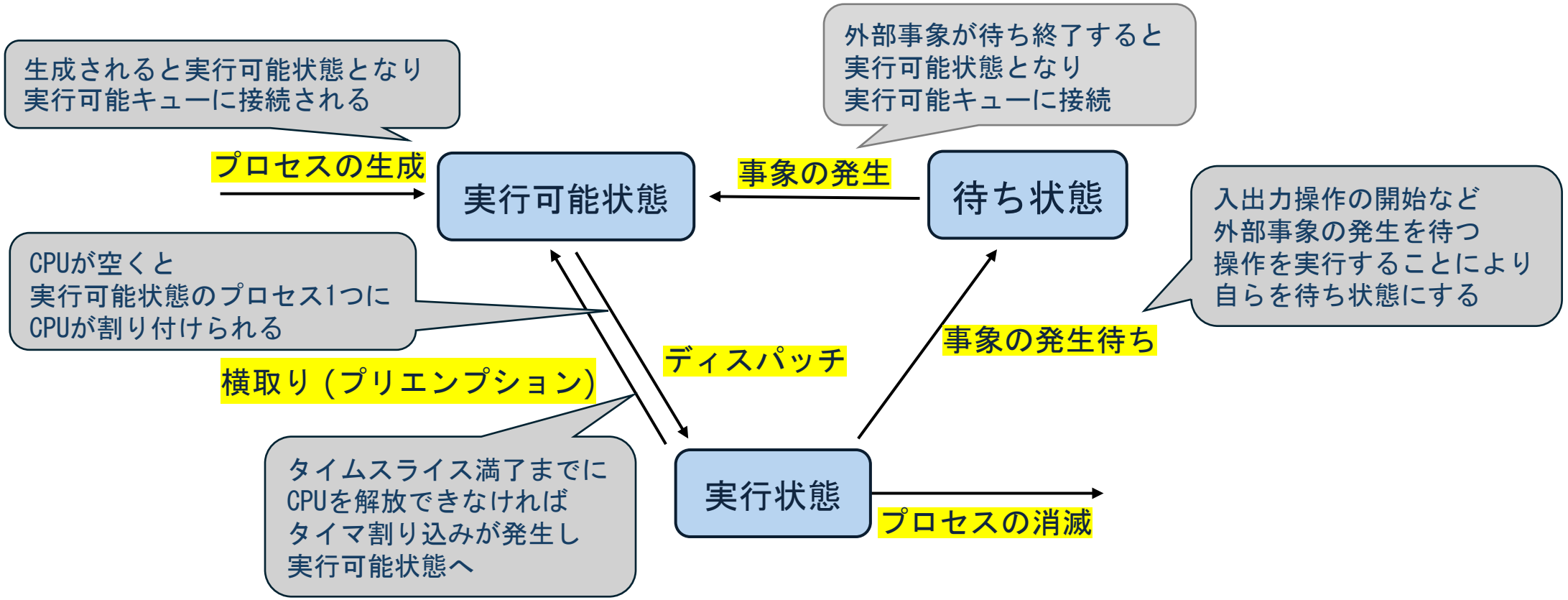
プロセスの状態

- プロセスは生成されてから消滅するまでに3つの状態間を遷移する
  - **実行可能状態** (ready state)
    - 他のプロセスがCPUを使っている間，CPU の割り当てを待っている状態
  - **実行状態** (running state)
    - CPU が割り当てられ，実行中の状態
  - **待ち状態** (waiting state)
    - 入出力の完了などの事象の発生を待っている状態

全てのプロセスはいずれかの状態にある



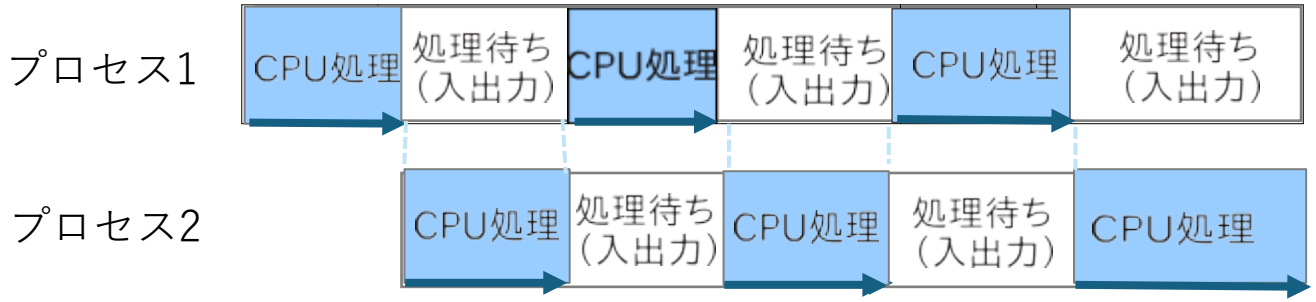
プロセスの状態遷移



タイムスライス (time slice) / タイムクウォンタム(time quantum):
プロセスが連続して走行可能な時間量.
特定のプロセスがCPUを占有し続けることを防止.

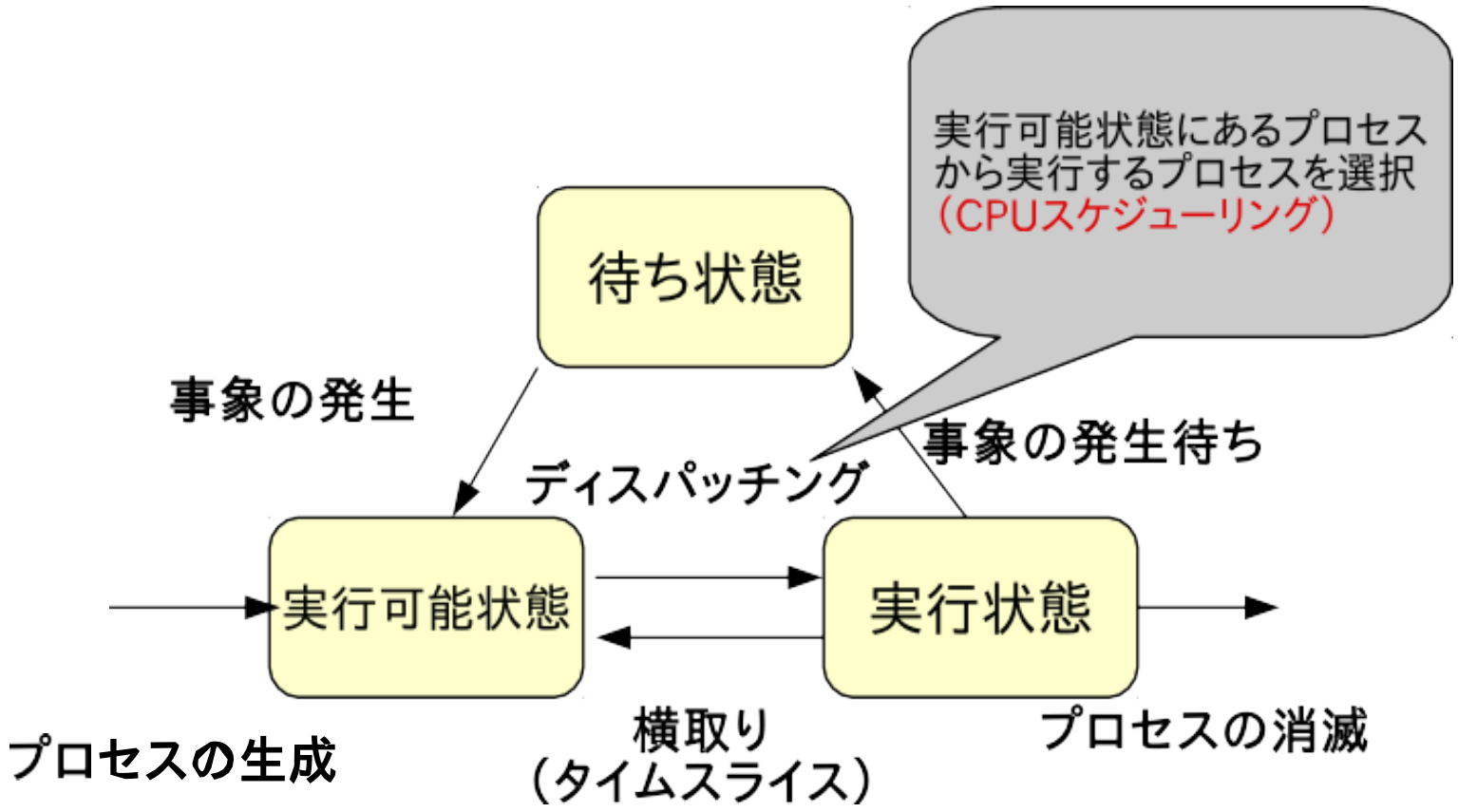
マルチプログラミングとプロセス

- プロセスはCPU処理と入出力処理が交互に表れる処理の系列
  - プロセスが入出力完了待ちになるとCPUはアイドル状態になる
  - マルチプログラミング (multi programming) でCPU利用率向上
    - 例) プロセス1と2をオーバーラップして実行することで50% → 100%に



  - 実際は上記のように都合良くはいかない
    - CPUバウンド (長いCPU処理)，入出力バウンド (長いI/O処理)のプロセス
    - 適切なスケジューリングでCPU利用率を上げることができる

プロセスの状態遷移とCPUスケジューリング



CPUスケジューラ

- **CPUスケジューラ（CPU scheduler）:**
  - 実行可能状態にあるプロセスの集合からCPUを割り付けるプロセスを選択
  - システム内に到着したプロセスをキューにつなぎ，プロセス数を制限
  - システム内のプロセス数 → マルチプログラミングの多重度

- **ディスパッチャ（dispatcher）:**
  - CPUスケジューラから呼び出される
  - 選択されたプロセスに実際にCPUを割り付ける
    - 実行中のプロセスのレジスタを退避，次に実行するプロセスのレジスタを回復

CPUスケジューリングの基準

- CPU 利用率の向上
  - システム稼働時間に対するCPUの動作時間
- スループット (処理効率)の向上
  - CPU が単位時間あたりに実行する仕事量
- **ターンアラウンド時間**
  - プロセスの実行を要求してから終了までの時間
- 待ち時間
  - プロセスが完了するまでに実行キューで待つ時間の合計
- 応答時間
  - 対話的な環境で応答が返ってくるまでの時間
- 公平な割り当て

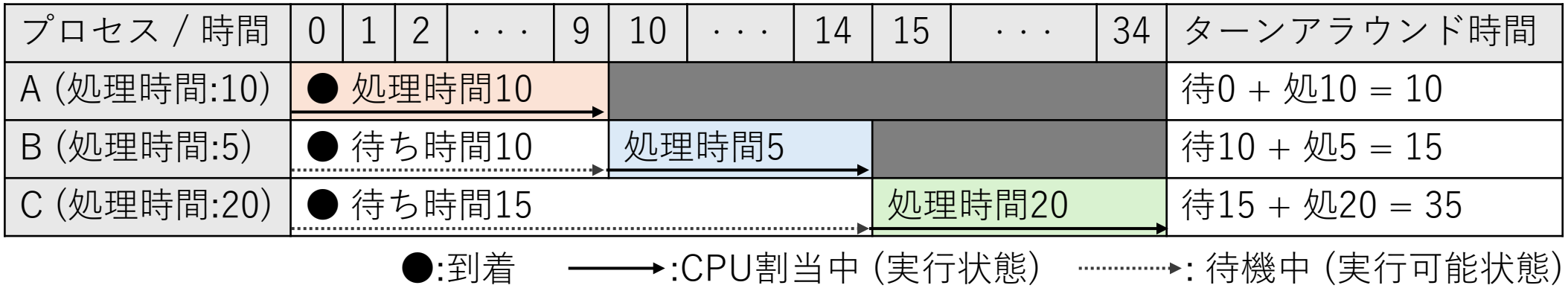
良いスケジューリングであるかどうかは，何を重視するかによって変わる．
例) バッチ処理ではターンアラウンド時間，対話処理では応答時間と公平性を重視

CPUスケジューリングの分類

- **横取り（preemption）:**
  - プロセス実行中にCPU を奪い取ること
- 横取り不可能なスケジューリング(non-preemptive Scheduling)
  - 実行中のプロセスがCPUを解放するまで他のプロセスに割り付け不可
  - 例）FCFS，SJFスケジューリング
- 横取り可能なスケジューリング(preemptive Scheduling)
  - 実行中のプロセスからCPUを奪い取り他のプロセスに割り付け可能
  - 例）優先度，ラウンドロビンスケジューリング

FCFSスケジューリング

- **到着順 (FCFS: First Come First Serve)**
  - 実行可能キューへの到着順 (FIFO: First In First Out)でCPUを割り付ける
    - プロセスが実行可能状態になると対応するPCBが実行可能キューの最後につながれる
    - 例) プロセスA (処理時間:10) → B (5) → C (20)の順で全て時刻0に到着した場合の平均ターンアラウンド時間は (10 + 15 + 35) / 3 = 20
    - 例) プロセスC (処理時間:20) → A (10) → B (5)の順で全て時刻0に到着した場合の平均ターンアラウンド時間は (20 + 30 + 35) / 3 = 28.333…



処理時間の長いものが先に到着すると，平均ターンアラウンド時間が長くなる

SJFスケジューリング

- **最短時間順 (SJF: Shortest Job First)**
  - 最も短いCPU処理を持つプロセスから順にCPUを割り当てる
    - 平均待ち時間・平均ターンアラウンド時間を 小さくすることができる
    - 例)プロセスA (処理時間:10) → B (5) → C (20)の順で全て時刻0に到着した場合の
      平均ターンアラウンド時間は (15 + 5 + 35) / 3 = 18.333… < 20 (FCFS)

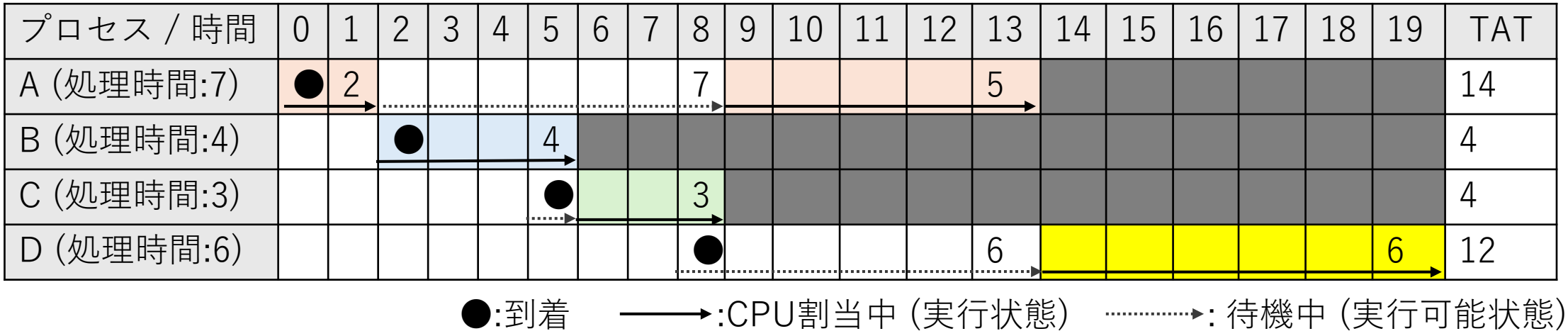
プロセス / 時間	0	1	2	3	4	5	・・・	14	15	・・・	34	ターンアラウンド時間
A (処理時間:10)	● 待ち時間5					処理時間10						5 + 10 = 15
B (処理時間:5)	● 処理時間5											0 + 5 = 5
C (処理時間:20)	● 待ち時間15								処理時間20			15 + 20 = 35

●:到着　——→:CPU割当中 (実行状態)　·····→: 待機中 (実行可能状態)

処理時間を事前に知ることはできないため現実のアルゴリズムとして使用されない

SRTFスケジューリング

- **残余処理時間順 (SRTF: Shortest Remaining Time First)**
  - 横取りありのSJFスケジューリング
  - 到着した**新しいプロセスの処理時間 < 実行中の残処理時間**ならば切り替え
  - 平均ターンアラウンド時間は (14 + 4 + 4 + 12) / 4 = 8.5



処理時間を事前に知ることはできないため現実のアルゴリズムとして使用されない
→ 経験則から近似的に処理時間を求めてスケジューリングを行っている

(p40の2.9) SJFとSRTFの比較

- SJFの場合: 平均ターンアラウンド時間は (7 + 12 + 5 + 12) / 4 = 9

プロセス / 時間	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	TAT
A (処理時間:7)	●						7														7
B (処理時間:4)			●							8				4							12
C (処理時間:3)						●	2			3											5
D (処理時間:6)									●					6						6	12

- SRTFの場合: 平均ターンアラウンド時間は (14 + 4 + 4 + 12) / 4 = 8.5

プロセス / 時間	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	TAT
A (処理時間:7)	●	2							7					5							14
B (処理時間:4)			●			4															4
C (処理時間:3)						●			3												4
D (処理時間:6)									●					6						6	12

●:到着　　→:CPU割当中 (実行状態)　　⇢: 待機中 (実行可能状態)

優先度スケジューリング

- **優先度（priority）スケジューリング:**
  - 各プロセスに優先度が割り付けられ，
    各時点で最も高い優先度のプロセスにCPUが割り付けられる
    - 優先度：デッドライン，主記憶の使用数，CPU処理時間，入出力時間などによる
    - 実行中のプロセスより優先度の高いプロセスが到着 → 横取り
    - 同じ優先度のプロセスはFCFSなどでスケジュール
  - 問題点：
    - 優先度の低いプロセスを無限に待たせる場合がある（無限の延期，無限の閉塞，飢餓）
  - 対処：エージング（aging）
    - 長時間システムに滞在しているプロセスの優先度を徐々に上げる

ラウンドロビンスケジューリング

- **ラウンドロビン（round-robin）スケジューリング**
  - タイムシェリングシステムのために設計
    - **一定時間（タイムスライス）**毎にFIFOキュー上のプロセスを順に切り替え実行
    - 例) 平均ターンアラウンド時間は (23 + 17 + 35) / 3 = 25 > 20 (FCFS)

プロセス / 時間	0	1	2	3	4	5	6	7	8	・・・	11	12	・・・	15	16	17	・・・	20
A (処理時間:10)	● 処理時間4				待ち時間8							処理時間4			待ち時間5			
B (処理時間:5)	● 待ち時間4				処理時間4				待ち時間8						1			
C (処理時間:20)	● 待ち時間8								処理時間4			待ち時間5				処理時間4		

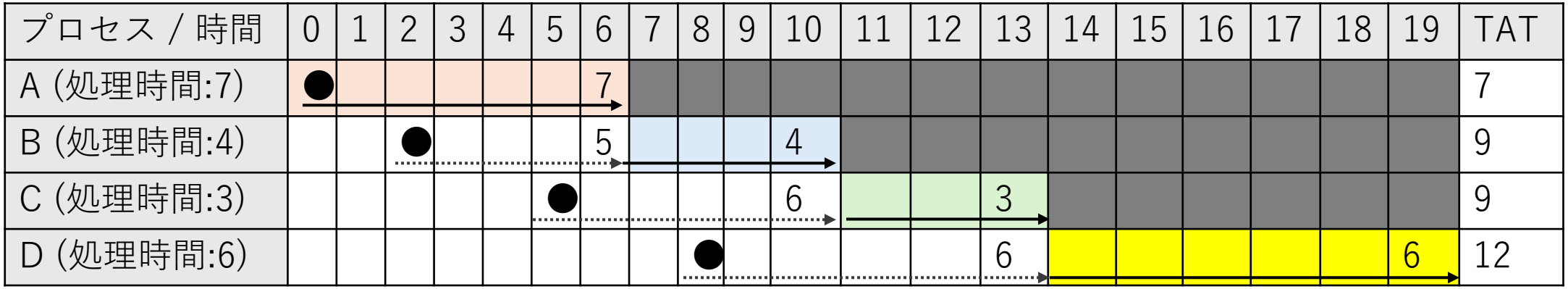
★タイムスライス = 4

21	22	23	・・・	26	27	・・・	30	31	・・・	34	ターンアラウンド時間
処理時間2											待13 + 処10 = 23
											待12 + 処5 = 17
待ち時間2		処理時間4			処理時間4			処理時間4			待15 + 処20 = 35

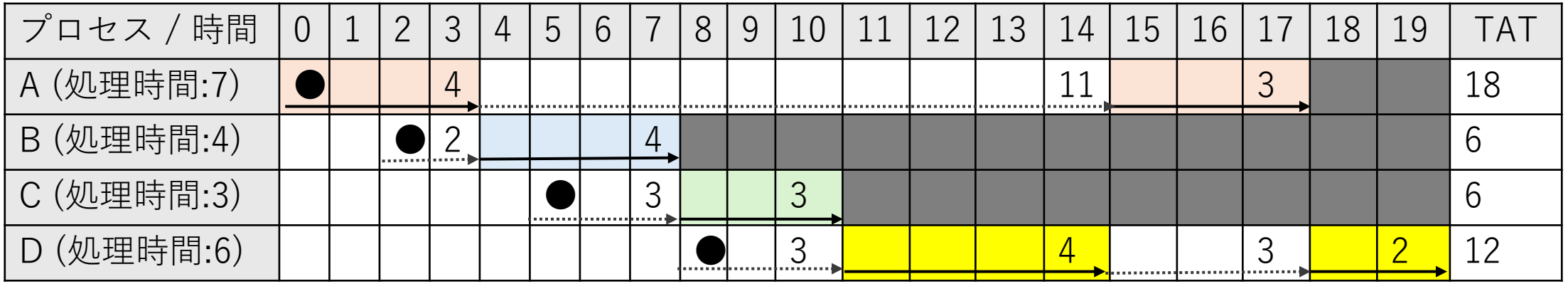
タイムスライスが短いとプロセスが頻繁に切り替わり，文脈切り替えのオーバーヘッドが無視できなくなる．極端に長いとFCFSと同じになる．

(p40の2.9) FCFSとラウンドロビンの場合

- FCFSの場合: 平均ターンアラウンド時間は (7 + 9 + 9 + 12) / 4 = 9.25



- ラウンドロビンの場合: 平均ターンアラウンド時間は (18 + 6 + 6 + 12) / 4 = 10.5



多重レベルスケジューリング

- **多重レベルスケジューリング (multi-level scheduling):**
  - プロセスを用途で分類し，複数の実行可能キューに振り分ける
  - 各キューに異なるスケジューリングアルゴリズムを適用：
    - リアルタイム処理：優先度スケジューリング
    - 会話型処理：ラウンドロビンスケジューリング
    - バッチ処理：FCFS スケジューリング
  - プロセスは１つのキューに固定的に割り付けられ，キュー間で移動しない

  - **多重レベルフィードバックスケジューリング (multi-level feedback scheduling) :**
    - CPUを長く占有しているプロセスをより優先度の低いキューに移動させる (キュー間で移動あり)

スレッドと軽量プロセス

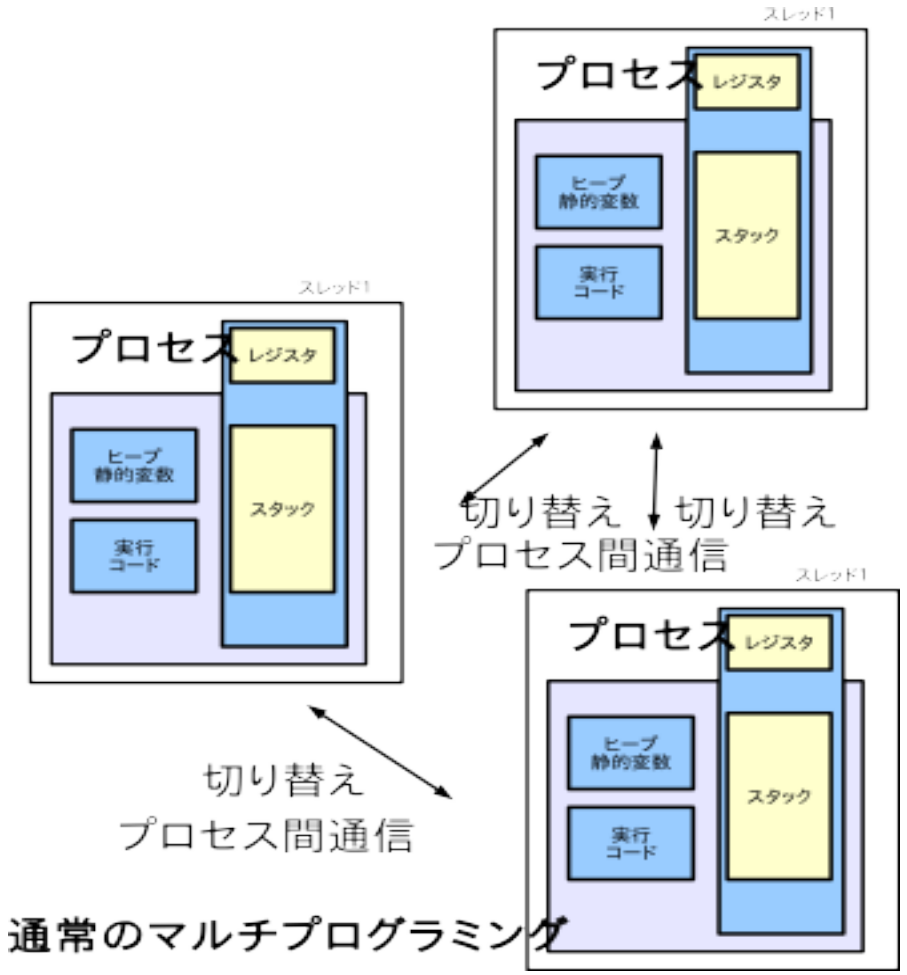
- 通常のプロセス切り替えの問題点
  - プロセスの生成，切り替えのオーバヘッドが大きい
  - 生成時にメモリの確保が必要
  - プロセス間でのデータのやりとりにプロセス間通信が必要

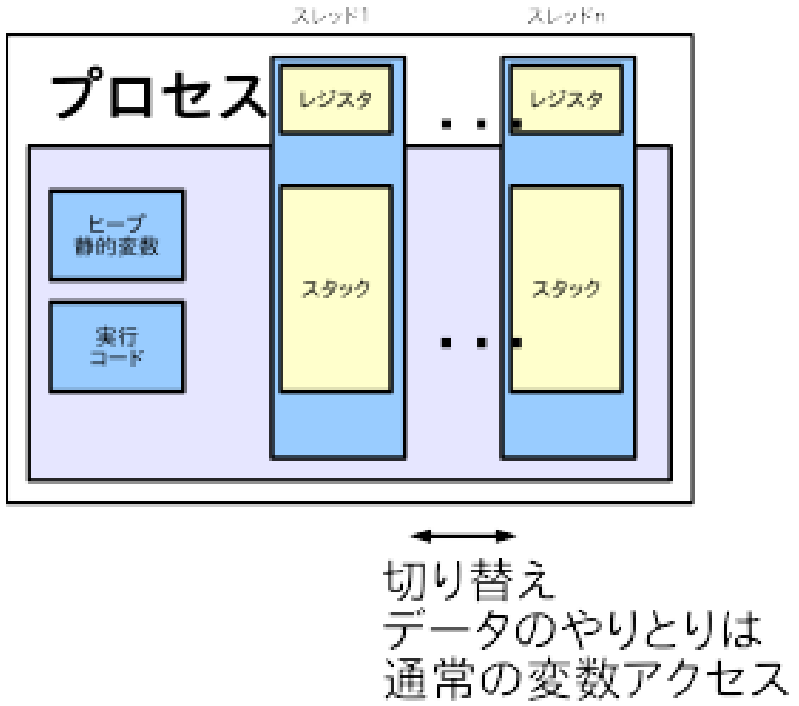
- **スレッド (thread) / 軽量プロセス (light-weight process):**
  - １つのプロセスの中に複数の制御の流れ (並行処理) を実現
    - CPU処理と入出力処理のオーバーラップ
    - 並列アルゴリズムを容易に表現できる
  - スレッドの生成は，プロセスの生成より 10 倍から100 倍高速
  - アドレス空間などプロセス内の資源を共有（プロセス間通信不要）
  - 応答性の向上 (短時間における大量の処理の応答を短縮する場合特に有効)
    - 例) Web サーバ，データベースサーバ

スレッドと軽量プロセス (続き)

- スレッドはプロセスそのものではない
  - 制御の流れを表し実行に必要な最小の文脈 (レジスタとスタック)をもつ
  - 通常のプロセスを重量プロセスと呼ぶこともある
- プロセスの文脈での情報
  - アドレス空間，大域変数，オープンしたファイル，子プロセス，タイマ，シグナル，セマフォ，アカウント情報など
- スレッドに固有の情報
  - プログラムカウンタ，レジスタセット，スタック，子スレッド，状態
  - スレッドに固有な情報以外は各スレッドで共有

プロセスとスレッド


スレッド1
プロセス
レジスタ
ヒープ
静的変数
実行
コード
スタック
切り替え
プロセス間通信
切り替え
プロセス間通信
通常のマルチプログラミング


スレッド1
スレッドn
プロセス
レジスタ
レジスタ
ヒープ
静的変数
実行
コード
スタック
スタック
切り替え
データのやりとりは
通常の変数アクセス
マルチスレッド
コンテキスト切り替えが不要
プロセス間通信も不要

スレッドとプロセスの違いを観察しよう（Pythonの場合）

- 付録（example_3.pyを実行して確認しましょう）

項目	プロセス	スレッド
メモリ空間	独立（親子で共有しない）	共有（親子で同じ）
グローバル変数	共有しない	共有する
並行性	マルチコア活用可	GILの制約あり
用途	CPUバウンド処理	I/Oバウンド処理

- スレッドはメモリを共有する → 競合に注意（Lockが必要）
- プロセスはメモリが独立 → 安全だがコストが高い

- GIL（Global Interpreter Lock）とは？
  Pythonのインタープリタが同時に1つのスレッドだけを実行できるようにする仕組み.
  → CPUバウンド処理では並列化できない
  → I/O待ち中は他スレッドが実行される

【参考】C言語のソースプログラムの例

(sample.c)

```
#include <stdio.h>
#include <string.h>

int main(void)
{
    int i;
    char *str = "String";

    for(i=0; i<strlen(str); i++) {
        printf("str[%d] = %c\n", i, str[i]);
    }

    return 0;
}
```

【参考】コンパイル結果の例 (Assembler)

(gcc –S sample.c –o sample.s で出力)

```
        .file   "sample.c"
        .text
        .section        .rodata
.LC0:
        .string "String"
.LC1:
        .string "str[%d] = %c\n"
        .text
        .globl  main
        .type   main, @function
main:
        endbr64
        pushq   %rbp
        movq    %rsp, %rbp
        pushq   %rbx
        subq    $24, %rsp
        leaq    .LC0(%rip), %rax
        movq    %rax, -24(%rbp)
        movl    $0, -28(%rbp)
        jmp     .L2
```

```
.L3:
        movl    -28(%rbp), %eax
        movslq  %eax, %rdx
        movq    -24(%rbp), %rax
        addq    %rdx, %rax
        movzbl  (%rax), %eax
        movsbl  %al, %edx
        movl    -28(%rbp), %eax
        movl    %eax, %esi
        leaq    .LC1(%rip), %rax
        movq    %rax, %rdi
        movl    $0, %eax
        call    printf@PLT
        addl    $1, -28(%rbp)
.L2:
        movl    -28(%rbp), %eax
        movslq  %eax, %rbx
        movq    -24(%rbp), %rax
        movq    %rax, %rdi
        call    strlen@PLT
        cmpq    %rax, %rbx
        jb      .L3
        movl    $0, %eax
        movq    -8(%rbp), %rbx
        leave
        ret
        .size   main, .-main
        .ident  "GCC: (Ubuntu 11.4.0-1ubuntu1~22.04) 11.4.0
        .section        .note.GNU-stack,"",@progbits
        .section        .note.gnu.property,"a"
```

【参考】ブラウザ上でコンパイルを試してみよう

- ブラウザ上のツール (COMPILER EXPLORER) で気軽に試せる

